

Sampling round complexity of diffusion language models in regular languages

Jiarui Zhang
Student ID: 2023010875
jiarui-z23@mails.tsinghua.edu.cn *

Summer Research Report, 2026

Abstract

Diffusion language models update many token positions in parallel. We study the number of rounds needed to generate the input-output pair $(X, f_L(X))$, where X is uniform and f_L recognizes a fixed regular language.

Our main technical result characterizes finite Markov chains whose complete trajectories can be sampled from fair bits: this is possible for every horizon exactly when all trajectory probabilities are dyadic. Applying the result to a DFA gives a randomized- AC^0 sampler for every fixed binary regular language. As a corollary in the randomized- AC^0 regime, two rounds suffice with revision; without revision, a finite-monoid product tree gives $O(\log n / \log \log n)$ rounds, with a matching lower bound for regular languages outside AC^0 . The results can be extended to the Transformer regime under the equivalence between poly-width, finite-precision, finite-depth transformers and AC^0 circuits.

1 Introduction

Autoregressive language models expose an inherently sequential sampling interface: the distribution of the next token is evaluated only after the previous token has been fixed. Diffusion language models (DLMs), by contrast, repeatedly update many coordinates in parallel. Their practical appeal motivates a basic complexity question: how many parallel rounds are needed to sample a prescribed distribution?

This question is meaningful only after restricting the computation performed within a round. Motivated by the connection between AC^0 and constant-precision Transformers established by Li et al. [5], we first work in the randomized- AC^0 regime: each predictor is a randomized- AC^0 sampler, while the no-revision policy is an AC^0 function. A D -round sampler in this regime unrolls into a randomized Boolean circuit of depth $O(D)$. In section 6, we transfer the lower and upper bounds to the specified Transformer regime.

Our target distributions are generated by fixed binary regular languages. For $L \subseteq \{0, 1\}^*$ and $f_L(x) = \mathbf{1}[x \in L]$, let

$$\text{Pair}_{L,n} = (X, f_L(X)), \quad X \sim \text{Unif}(\{0, 1\}^n). \quad (1)$$

The sampler must jointly produce a uniform input and its correct membership bit. Parity is the canonical example: recognition is outside AC^0 , whereas its input-output pair distribution has a randomized- AC^0 local sampler, see algorithm 1; each output bit depends on at most two fair bits, so the underlying process is even randomized- NC^0 .

The second modeling decision is whether previously revealed tokens are immutable. A no-revision DLM only replaces masks by visible symbols. A DLM with revision may overwrite or remask visible coordinates, so the output positions themselves provide reusable workspace. Jiang et al. [3] formalize closely related parallel-sampling models and show that revision or remasking can strictly increase their power. We focus on fixed-length generation with no extra chain-of-thought or padded coordinates.

*Mentor: Jingzhao Zhang at jingzhaoz@mail.tsinghua.edu.cn

Algorithm 1 Parity pair sampler in constant depth

```
1: sample  $u_1, \dots, u_n \sim \text{Unif}(\{0, 1\})$  independently
2:  $X_1 \leftarrow u_1$ 
3: for  $i = 2, \dots, n$  do
4:    $X_i \leftarrow u_{i-1} \oplus u_i$ 
5:  $C \leftarrow u_n \oplus 1$   $\triangleright C = 1$  iff  $\bigoplus_i X_i = 0$ 
6: return  $(X, C)$ 
```

Contributions. The report integrates two parts of the project into one self-contained chain of results.

1. Finite Markov-chain trajectories with dyadic probabilities admit randomized- AC^0 samplers for every horizon; the dyadic condition is also necessary.
2. Revision gives better round complexity than no-revision in both the randomized- AC^0 and Transformer regimes: two rounds suffice with revision, while no-revision requires $\Theta(\log n / \log \log n)$ rounds for languages outside AC^0 under the stated model.

2 Related Work

Transformers and circuit complexity. Formal Transformer models have been bounded or characterized using classical circuit classes. Under a constant-precision, constant-depth, polynomial-width model, Li et al. [5] give the AC^0 connection used here. Other choices lead to different classes: saturated attention over floating-point values, for example, admits a TC^0 upper bound [6]. Svete et al. [7] study the reasoning power of diffusion language models and their relation to looped and padded Transformer computation.

DLMs and parallel sampling. The theory of diffusion language models increasingly studies generation as a sequence of parallel conditional updates. Jiang et al. [3] connect DLM rounds to parallel sampling circuits, establish simulations with chain-of-thought workspace, and show that revision or remasking enables workspace reuse. Our question is narrower: the output has exactly $n + 1$ positions, sampling is exact, and each round is restricted to randomized- AC^0 .

Complexity of distributions. The complexity of sampling a distribution can differ sharply from computing a function that describes its support. Viola developed a systematic circuit view of distributions [9]: parity and inner-product pairs $(X, b(X))$ are generated *exactly* by small randomized- AC^0 circuits (for parity, each output bit depends on at most two fair bits, so the underlying process is even randomized- NC^0), while for every symmetric function b such as majority or MOD_3 , randomized- AC^0 circuits generate $(X, b(X))$ only approximately, up to an exponentially small statistical distance. Later work constructs an explicit Boolean function h such that small randomized- AC^0 circuits cannot even approximately sample $(U, h(U))$ to small statistical distance [10].

Regular languages and shallow circuits. The algebraic theory of regular languages characterizes membership in AC^0 through aperiodicity of the stable syntactic monoid [1]. Classical random-generation work constructs logspace-uniform probabilistic circuits that generate words of $L \cap \Sigma^n$ exactly uniformly in polylogarithmic depth, up to a bounded rejection probability [2]. These results provide useful neighboring generation models. Our target equation (1) instead retains the uniform input word and appends its membership label.

Random maps and Markov chains. A finite Markov chain can be represented by random maps, as in road-coloring constructions [4, 11]; our overlap with this line of work is only a coloring view. Our objective is different: a fixed linear number of fair bits, a constant-depth circuit, and the whole trajectory at once.

3 Models and Preliminaries

3.1 Randomized constant-depth circuits

AC^0 is the class of Boolean functions computed by polynomial-size, constant-depth circuits with unbounded-fan-in AND and OR gates and NOT gates. A randomized- AC^0 sampler is such a circuit whose inputs are independent fair bits. A family $C_n : \{0, 1\}^{r(n)} \rightarrow \Omega_n$ samples a distribution $\mathcal{D}_n$ exactly when

$$\Pr[C_n(U_{r(n)}) = z] = \Pr_{Z \sim \mathcal{D}_n}[Z = z] \quad \text{for every } z \in \Omega_n. \quad (2)$$

Exactness immediately imposes an arithmetic constraint: every output atom of a finite fair-bit sampler is dyadic, i.e., of the form $m/2^k$.

3.2 DLM update rules

Fix the vocabulary $V = \{0, 1\}$ and a mask symbol $\mathbf{M}$. A sequence of length N begins at $\mathbf{M}^N$. In the no-revision model, a policy F selects some currently masked coordinates to unmask and a predictor p supplies the marginal distributions. Given the current state, selected coordinates are sampled independently:

Algorithm 2 No-revision DLM

```

1:  $z \leftarrow \mathbf{M}^N$ 
2: for  $t = 1, \dots, D$  do
3:    $S \leftarrow F(z)$ , where  $S \subseteq \{i : z_i = \mathbf{M}\}$ 
4:   for all  $i \in S$  in parallel do
5:     sample  $z_i \sim p_i(\cdot \mid z)$  over  $\{0, 1\}$ 
6: return  $z$ 
```

Algorithm 3 DLM with revision

```

1:  $z \leftarrow \mathbf{M}^N$ 
2: for  $t = 1, \dots, D$  do
3:   for all  $i \in [N]$  in parallel do
4:     sample  $z_i \sim p_i(\cdot \mid z)$  over  $\{0, 1\} \cup \{\mathbf{M}\}$ 
5: return  $z$ 
```

With revision, every coordinate may be updated to 0, 1, or $\mathbf{M}$ in every round. A valid sampler for a binary target returns no mask with probability one. In both models, each local predictor is a randomized- AC^0 sampler: it receives the current state and fresh independent fair bits and outputs one token. The random bits used at different updated coordinates are independent. The no-revision policy F is an AC^0 circuit and has no random input. In the Transformer realization, the backbone computes the predictor's logits and the randomized local sampler is obtained by independently decoding from them.

3.3 Automata and aperiodicity

For a DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$, every word $u \in \Sigma^*$ induces a map on its states,

$$T_u(q) = \delta^*(q, u). \quad (3)$$

The set

$$M_{\mathcal{A}} = \{T_u : u \in \Sigma^*\} \quad (4)$$

is the transition monoid of $\mathcal{A}$, and $\eta_{\mathcal{A}} : \Sigma^* \rightarrow M_{\mathcal{A}}$, defined by $\eta_{\mathcal{A}}(u) = T_u$, is its transition morphism.

Definition 3.1 (Stable monoid). A positive integer d is a *stability index* of a morphism $\eta : \Sigma^* \rightarrow M$ if

$$\eta(\Sigma^d) = \eta(\Sigma^{2d}). \quad (5)$$

When stability indices exist, let d_η be the smallest one. The *stable monoid* of η is

$$\text{Stab}(\eta) = \eta((\Sigma^{d_\eta})^*) = \{1\} \cup \eta(\Sigma^{d_\eta}). \quad (6)$$

A stability index always exists when M is finite. Indeed, the sequence of sets $\eta(\Sigma^j)$ is eventually periodic: for some $d_0, \ell \geq 1$, $\eta(\Sigma^{d_0}) = \eta(\Sigma^{d_0+\ell})$. Multiplication by $\eta(\Sigma^k)$ gives the same equality after adding any $k \geq 0$ to both lengths. Taking $d = \lceil d_0/\ell \rceil \ell$ therefore yields $\eta(\Sigma^d) = \eta(\Sigma^{2d})$. Thus d_η is well-defined. Applying (5) to d_η also makes $\eta(\Sigma^{d_\eta})$ closed under multiplication, which gives the second equality in (6).

A finite monoid is *aperiodic* if every element a satisfies $a^{k+1} = a^k$ for some $k \geq 1$. Here and below, a nontrivial cycle means a directed cycle of length at least 2.

Lemma 3.2 (Cycle criterion). *Let $g : H \rightarrow H$ be a self-map of a finite set. Then $g^k = g^{k+1}$ for some $k \geq 1$ if and only if g has no nontrivial cycle. Consequently, a finite transformation monoid is aperiodic if and only if none of its elements has a nontrivial cycle.*

Proof. If g has a nontrivial cycle, then $g^m \neq g^{m+1}$ for every $m \geq 1$. Conversely, if every cycle of g is a fixed point, then every orbit reaches a fixed point within $|H|$ steps, so $g^{|H|} = g^{|H|+1}$. $\square$

Lemma 3.3 (AC^0 recognition characterization). *Let $\mathcal{A}$ be the minimal DFA of a regular language L . Then $L \in \text{AC}^0$ if and only if the stable monoid $\text{Stab}(\eta_{\mathcal{A}})$ is aperiodic.*

This is the Barrington–Compton–Straubing–Thérien characterization [1]. For a minimal DFA, the transition monoid is the syntactic monoid, so $\text{Stab}(\eta_{\mathcal{A}})$ is also called the stable syntactic monoid of L .

Lemma 3.4 (Aperiodic transition monoid implies AC^0). *If the full transition monoid $M_{\mathcal{A}}$ of a DFA is aperiodic, then the language recognized by $\mathcal{A}$ is in AC^0 .*

Proof. Minimize $\mathcal{A}$. The transition monoid of the minimal DFA is a quotient of $M_{\mathcal{A}}$. A quotient of an aperiodic monoid is aperiodic, and every submonoid is also aperiodic. Thus the stable monoid of the minimal DFA is aperiodic, and lemma 3.3 applies. We use this implication to compute all prefix states of an aperiodic block process in parallel. $\square$

Lemma 3.5 (Monotone DFA test). *Suppose the states of a fixed DFA have a total order $\leq$. If every character preserves this order, meaning that*

$$x \leq y \implies \delta(x, c) \leq \delta(y, c) \quad (7)$$

for all states x, y and characters $c \in \Sigma$, then the language of the DFA is in AC^0 .

Proof. Each character induces an order-preserving map, and a composition of such maps is still order-preserving. Therefore every word $w \in \Sigma^*$ induces an order-preserving map T_w .

Fix a state x and define $x_i = T_w^i(x)$. Because the order is total, either $x_1 \leq x_0$ or $x_1 \geq x_0$. In the first case, order preservation gives

$$x_{i+1} = T_w(x_i) \leq T_w(x_{i-1}) = x_i, \quad (8)$$

so the sequence keeps decreasing. In the second case, the same argument shows that it keeps increasing. Since Q is finite, the sequence must become constant. Thus, for every $x \in Q$, there is an integer ℓ_x such that

$$T_w^{\ell_x}(x) = T_w^{\ell_x+1}(x). \quad (9)$$

Let $N_w = \max_{x \in Q} \ell_x$. Then

$$T_w^{N_w} = T_w^{N_w+1}. \quad (10)$$

$M_{\mathcal{A}}$ is aperiodic since w was arbitrary, and lemma 3.4 applies. $\square$

4 Revision: Exact Sampling and Two Rounds

Fix a finite Markov chain

$$\mathcal{M} = (Q, \mu, P), \quad (11)$$

where μ is the initial distribution on Q and P is the transition matrix.

For a length n , a trajectory $\gamma = (x_0, \dots, x_n) \in Q^{n+1}$ has probability

$$p_{\mathcal{M}}(\gamma) = \mu(x_0) \prod_{i=1}^n P(x_{i-1}, x_i). \quad (12)$$

Write $\text{Path}_n(\mathcal{M})$ for the distribution on Q^{n+1} defined by

$$\Pr_{\gamma \sim \text{Path}_n(\mathcal{M})} [\gamma = (x_0, \dots, x_n)] = p_{\mathcal{M}}(x_0, \dots, x_n). \quad (13)$$

Let $\mathbb{D}_{\geq 0}$ be the set of nonnegative dyadic values. Call $\mathcal{M}$ *transition-dyadic* when

$$\mu(q), P(q, r) \in \mathbb{D}_{\geq 0} \quad \text{for all } q, r \in Q. \quad (14)$$

Call $\mathcal{M}$ *path-dyadic from μ* when every trajectory atom is dyadic:

$$p_{\mathcal{M}}(\gamma) \in \mathbb{D}_{\geq 0} \quad \text{for every } n \geq 0 \text{ and } \gamma \in Q^{n+1}. \quad (15)$$

Transition-dyadicity is sufficient but not necessary. For example, on states $\{a, b, c\}$, let

$$\mu = \left(\frac{3}{4}, \frac{1}{4}, 0 \right), \quad P = \begin{pmatrix} 0 & 1/3 & 2/3 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}. \quad (16)$$

The transitions $1/3$ and $2/3$ are not dyadic, but for every $n \geq 1$ the only positive trajectory atoms have probabilities

$$\frac{3}{4} \cdot \frac{1}{3} = \frac{1}{4}, \quad \frac{3}{4} \cdot \frac{2}{3} = \frac{1}{2}, \quad \frac{1}{4}. \quad (17)$$

Thus the chain is path-dyadic.

4.1 Transition-dyadic trajectories

Lemma 4.1 (Finite-state convergence). *Let K be an irreducible and aperiodic transition matrix on a finite state space. Then K has a unique stationary distribution π , with $\pi(x) > 0$ for every state x . For every $\varepsilon > 0$, there is m_{ε} such that, for every initial distribution λ and every $m \geq m_{\varepsilon}$,*

$$\|\lambda K^m - \pi\|_{\text{TV}} < \varepsilon. \quad (18)$$

See Levin and Peres, with contributions by Wilmer, *Markov Chains and Mixing Times*, Chapter 1 and Theorem 4.9.

Proposition 4.2 (Ordered block-census sampling criterion). *Let $\mathcal{A} = (Q, \{0, 1\}, \delta, q_0, Q_{\text{acc}})$ be a fixed binary DFA, and let $L = L(\mathcal{A})$. Fix a block length $b \geq 1$ and a total order*

$$Q = \{\xi_0 < \xi_1 < \dots < \xi_{\ell-1}\}. \quad (19)$$

Let $\Gamma = \{0, 1\}^b$. For $s, t \in Q$, define the block-transition census

$$M_s(t) = |\{y \in \Gamma : \delta^*(s, y) = t\}|. \quad (20)$$

For $\text{Pfx}_k = \{\xi_0, \dots, \xi_k\}$, set

$$F_{s, \text{Pfx}_k} = \sum_{t \in \text{Pfx}_k} M_s(t). \quad (21)$$

Assume that, for all $0 \leq i < \ell - 1$ and $0 \leq k < \ell$,

$$F_{\xi_i, \text{Pfx}_k} \geq F_{\xi_{i+1}, \text{Pfx}_k}. \quad (22)$$

Then there is a randomized- AC^0 sampler for $\text{Pair}_{L, n}$.

Proof. Let $N = 2^b$ and set $F_{s, \text{Pfx}_{-1}} = 0$. Order Γ lexicographically. We use the discrete common-quantile, or monotone, coupling [8, Chapter 1, Section 3]: the same rank is passed through the inverse cumulative census for every initial state. For $u \in \Gamma$, define $\tau_u(s) = \xi_k$ when

$$F_{s, \text{Pfx}_{k-1}} \leq \text{rank}(u) < F_{s, \text{Pfx}_k}. \quad (23)$$

Consider adjacent states $s = \xi_i < s' = \xi_{i+1}$. If $\tau_u(s') = \xi_t$, then

$$\text{rank}(u) < F_{s', \text{Pfx}_t} \leq F_{s, \text{Pfx}_t}. \quad (24)$$

Therefore $\tau_u(s) \leq \xi_t = \tau_u(s')$. Thus every τ_u preserves the state order.

If $v = \xi_k$, then the two lists used in the rank matching below have the same length:

$$|\{u \in \Gamma : \tau_u(s) = v\}| = F_{s, \text{Pfx}_k} - F_{s, \text{Pfx}_{k-1}} = M_s(v) = |\{y \in \Gamma : \delta^*(s, y) = v\}|. \quad (25)$$

Since the two sets have equal cardinality, we obtain block-cube permutations $\pi_s : \Gamma \rightarrow \Gamma$ such that

$$\delta^*(s, \pi_s(u)) = \tau_u(s). \quad (26)$$

Each π_s is hard-coded into the circuit as a lookup table.

The sampler parses $z \in \{0, 1\}^n$ as $(u[1], \dots, u[m], c)$ with $u[i] \in \Gamma$ and $c \in \{0, 1\}^r$, where $n = bm + r$. It iterates $s[0] \leftarrow q_0$, $y[i] \leftarrow \pi_{s[i-1]}(u[i])$, $s[i] \leftarrow \tau_{u[i]}(s[i-1])$, and returns $(y[1], \dots, y[m], c)$ together with the label $\mathbf{1}[\delta^*(s[m], c) \in Q_{\text{acc}}]$.

Every τ_u preserves the state order, so by lemma 3.5 the state reached after any block prefix is computable in AC^0 . Compute all these states in parallel; each output block is then a fixed lookup. Thus the sampling procedure is in randomized- AC^0 . $\square$

Theorem 4.3 (Transition-dyadic trajectories). *Every transition-dyadic Markov chain has a randomized- AC^0 sampler for $\text{Path}_n(\mathcal{M})$. It uses at most $v + sn$ fair bits, where v depends on μ and s depends on P .*

Proof. First consider only the directed support of P . Let

$$P_{\text{bool}}(q, t) = \mathbf{1}[P(q, t) > 0], \quad (27)$$

with powers taken over the Boolean semiring. Since the Boolean powers P_{bool}^j take only finitely many values, there are $d_0, \ell \geq 1$ with $P_{\text{bool}}^{d_0} = P_{\text{bool}}^{d_0+\ell}$; take a to be a multiple of ℓ with $a \geq d_0$. Then

$$P_{\text{bool}}^a = P_{\text{bool}}^{2a}. \quad (28)$$

Set $P_{\text{aperiodic}} := P_{\text{bool}}^a$; this Boolean matrix is idempotent.

Now choose $s \geq 1$ such that every $P(q, t)$ is an integer multiple of 2^{-s} , and set

$$P_N = 2^s P, \quad \Gamma_s = \{0, 1\}^s. \quad (29)$$

Thus P_N is an integer matrix whose rows sum to 2^s . For each q , assign exactly $P_N(q, t)$ characters of Γ_s to the transition $q \rightarrow t$. This defines

$$\delta : Q \times \Gamma_s \longrightarrow Q. \quad (30)$$

For a positive integer c , use blocks in $\Gamma_s^{ac} \cong \{0, 1\}^{sac}$. For a block word $w = w_1 \cdots w_{ac} \in \Gamma_s^{ac}$, define

$$\delta(q, w) = \delta(\cdots \delta(\delta(q, w_1), w_2) \cdots, w_{ac}). \quad (31)$$

Then

$$|\{w \in \Gamma_s^{ac} : \delta(q, w) = t\}| = P_N^{ac}(q, t) = 2^{sac} P^{ac}(q, t). \quad (32)$$

Its Boolean support is

$$P_{\text{bool}}^{ac} = P_{\text{aperiodic}}^c = P_{\text{aperiodic}}, \quad (33)$$

so its SCCs do not depend on c .

Let R be a terminal SCC of $P_{\text{aperiodic}}$. Write $R = \{t_1, \dots, t_k\}$. Split each state into two hidden copies, set

$$H_R = \{t_1^-, \dots, t_k^-, t_1^+, \dots, t_k^+\}, \quad (34)$$

and order them as $t_1^- < \cdots < t_k^- < t_1^+ < \cdots < t_k^+$. Let $\phi(t_i^-) = \phi(t_i^+) = t_i$ and list the hidden copies as $h_1 < \cdots < h_{2k}$. A block transfer of hidden states is encoded by a nonnegative integer census

$$N_{\text{hidden}, c} : H_R \times H_R \longrightarrow \mathbb{Z}_{\geq 0}. \quad (35)$$

It must first satisfy exact projection:

$$\sum_{\substack{h' \in H_R \\ \phi(h') = t}} N_{\text{hidden}, c}(h, h') = P_N^{ac}(\phi(h), t) \quad (h \in H_R, t \in R). \quad (36)$$

Summing over $t \in R$ gives a row sum of 2^{sac} .

The key idea is to extend the order-preserving structure of proposition 4.2, so we also require

$$\sum_{\substack{\tilde{h} \in H_R \\ \tilde{h} \leq x}} N_{\text{hidden}, c}(h_j, \tilde{h}) \geq \sum_{\substack{\tilde{h} \in H_R \\ \tilde{h} \leq x}} N_{\text{hidden}, c}(h_{j+1}, \tilde{h}) \quad (37)$$

for every $1 \leq j < 2k$ and $x \in H_R$.

Since $P_{\text{aperiodic}}|_R$ is the all-one matrix, lemma 4.1 applied to $P^a|_R$ gives a stationary distribution π_R and, for $1 \leq j \leq 2k$ and $1 \leq i \leq k$,

$$2^{-sac} P_N^{ac}(\phi(h_j), t_i) = P^{ac}(\phi(h_j), t_i) \longrightarrow \pi_R(t_i) \quad (c \longrightarrow \infty). \quad (38)$$

The construction is

$$\begin{aligned} N_{\text{hidden},c}(h_j, t_i^-) &= \left\lfloor \frac{2k+1-j}{2k+1} P_N^{ac}(\phi(h_j), t_i) \right\rfloor, \\ N_{\text{hidden},c}(h_j, t_i^+) &= \left\lceil \frac{j}{2k+1} P_N^{ac}(\phi(h_j), t_i) \right\rceil. \end{aligned} \quad (39)$$

These formulas apply for $1 \leq j \leq 2k$ and $1 \leq i \leq k$. The two integers sum exactly to $P_N^{ac}(\phi(h_j), t_i)$, so exact projection holds.

The rounding error in each integer count is at most 1. Therefore, for $1 \leq j < 2k$ and $1 \leq r \leq k$,

$$\begin{aligned} &2^{-sac} \sum_{i=1}^r (N_{\text{hidden},c}(h_j, t_i^-) - N_{\text{hidden},c}(h_{j+1}, t_i^-)) \\ &= \sum_{i=1}^r \left(\frac{2k+1-j}{2k+1} P^{ac}(\phi(h_j), t_i) - \frac{2k-j}{2k+1} P^{ac}(\phi(h_{j+1}), t_i) \right) + O(r2^{-sac}) \\ &\longrightarrow \frac{1}{2k+1} \sum_{i=1}^r \pi_R(t_i) > 0. \end{aligned} \quad (40)$$

For $1 \leq j < 2k$ and $1 \leq r < k$, exact row sums similarly give

$$\begin{aligned} &2^{-sac} \sum_{\substack{\tilde{h} \in H_R \\ \tilde{h} \leq t_r^+}} (N_{\text{hidden},c}(h_j, \tilde{h}) - N_{\text{hidden},c}(h_{j+1}, \tilde{h})) \\ &= 2^{-sac} \sum_{i=r+1}^k (N_{\text{hidden},c}(h_{j+1}, t_i^+) - N_{\text{hidden},c}(h_j, t_i^+)) \\ &= \sum_{i=r+1}^k \left(\frac{j+1}{2k+1} P^{ac}(\phi(h_{j+1}), t_i) - \frac{j}{2k+1} P^{ac}(\phi(h_j), t_i) \right) + O((k-r)2^{-sac}) \\ &\longrightarrow \frac{1}{2k+1} \sum_{i=r+1}^k \pi_R(t_i) > 0. \end{aligned} \quad (41)$$

All prefix-sum differences are positive, and there are only finitely many choices of j and r . Hence there is c_R such that every ordered-cut inequality holds for all $c \geq c_R$. There are only finitely many terminal SCCs, so one common $c \geq \max_R c_R$ works for all of them.

For $u \in \Gamma_s^{ac}$, let $\tau_u(h)$ be constructed as in proposition 4.2, so τ_u is order-preserving on each split terminal SCC. Consequently, every composition of these maps is also order-preserving there and has no nontrivial cycle.

For h in a terminal SCC and $t \in Q$, the sets

$$D_{h,t} = \{u \in \Gamma_s^{ac} : \phi(\tau_u(h)) = t\} \quad (42)$$

and

$$W_{h,t} = \{w \in \Gamma_s^{ac} : \delta(\phi(h), w) = t\} \quad (43)$$

have the same cardinality. Match them for every t . The union of these matches is a block-cube permutation

$$\pi_h : \Gamma_s^{ac} \longrightarrow \Gamma_s^{ac} \quad (44)$$

such that

$$\delta(\phi(h), \pi_h(u)) = \phi(\tau_u(h)). \quad (45)$$

We now rule out nontrivial cycles using the cycle criterion. Every hidden transition used above projects to an edge of $P_{\text{aperiodic}}$. The condensation graph of $P_{\text{aperiodic}}$ is acyclic, so every map stays in its current SCC or moves downstream. Therefore any nontrivial cycle of a composition of seed maps must lie entirely in one SCC.

It remains to prevent such nontrivial cycles inside transient SCCs. Let C be a transient SCC and keep only one hidden copy h_q of each $q \in C$, with $\phi(h_q) = q$. Since C is transient for P^a ,

$$\sum_{q \in C} P^{ac}(q, C) \longrightarrow 0. \quad (46)$$

Increase c further, if necessary, so that the terminal CDF order still holds and

$$\sum_{q \in C} P^{ac}(q, C) < 1 \quad (47)$$

for every transient SCC C . Equivalently,

$$\sum_{q \in C} \sum_{t \in C} P_N^{ac}(q, t) < 2^{sac}. \quad (48)$$

This allows us to assign a different seed to every internal transition $h_q \rightarrow h_t$, with $q, t \in C$. Thus each seed induces at most one transition inside C , and any composition does as well. The corresponding block-cube permutations are constructed in the same way.

By the cycle criterion, the seed-generated monoid is aperiodic.

Choose $v \geq 0$ such that every $\mu(q)$ is an integer multiple of 2^{-v} , and fix a lookup $g : \{0, 1\}^v \rightarrow Q$ whose output has law μ . Also fix one hidden copy $\iota(q) \in \phi^{-1}(q)$ for every q . The complete sampler is algorithm 4.

Algorithm 4 SamplePath: sampling a transition-dyadic trajectory

```

1: Input:  $st \in \{0, 1\}^v$  and  $z \in \{0, 1\}^{sn}$ 
2: write  $n = ac \cdot m + r$  with  $0 \leq r < ac$ 
3: parse  $z$  as  $(u[1], \dots, u[m], e[1], \dots, e[r])$ , with  $u[i] \in \Gamma_s^{ac}$  and  $e[j] \in \Gamma_s$ 
4:  $x[0] \leftarrow g(st)$  ▷ sample the initial visible state from  $\mu$ 
5:  $h[0] \leftarrow \iota(x[0])$  ▷ choose its fixed hidden copy
6: for  $i = 1, \dots, m$  do
7:    $y[i] \leftarrow \pi_{h[i-1]}(u[i])$ 
8:    $h[i] \leftarrow \tau_{u[i]}(h[i-1])$ 
9:   parse  $y[i]$  as  $(y[i, 1], \dots, y[i, ac])$ 
10:  for  $j = 1, \dots, ac$  do
11:     $x[(i-1)ac + j] \leftarrow \delta(x[(i-1)ac + j - 1], y[i, j])$ 
12:  for  $j = 1, \dots, r$  do
13:     $x[mac + j] \leftarrow \delta(x[mac + j - 1], e[j])$ 
14: return  $(x[0], \dots, x[n])$ 

```

By lemma 3.4, the hidden state reached after any block prefix is computable in AC^0 . Compute all block-prefix states in parallel; the transformed blocks, their constant-length internal paths, and the final remainder are fixed lookups. Thus the sampler is randomized- AC^0 . $\square$

4.2 A finite dyadic lift

Path-dyadic chains reduce to the preceding theorem by splitting states into finite fibers.

Theorem 4.4 (Dyadic projection). *Every fixed path-dyadic Markov chain is the coordinatewise projection of a fixed transition-dyadic Markov chain. More precisely, there are a finite chain $\widetilde{\mathcal{M}} = (\widetilde{Q}, \widetilde{\mu}, \widetilde{P})$ and a map $\phi : \widetilde{Q} \rightarrow Q$ with the following property. If $(\widetilde{X}_i)_{i \geq 0}$ is the Markov chain with initial distribution $\widetilde{\mu}$ and transition matrix $\widetilde{P}$, then, for every $n \geq 0$ and every $(x_0, \dots, x_n) \in Q^{n+1}$,*

$$\Pr[\phi(\widetilde{X}_0) = x_0, \dots, \phi(\widetilde{X}_n) = x_n] = p_{\mathcal{M}}(x_0, \dots, x_n) = \mu(x_0) \prod_{i=1}^n P(x_{i-1}, x_i). \quad (49)$$

Proof. Assume every state is reachable with positive probability from μ . Otherwise, discard the unreachable states; this does not change the trajectory law.

We first construct positive integer weights $(w_q)_{q \in Q}$ satisfying

$$\frac{\mu(q)}{w_q} \in \mathbb{D}_{\geq 0}, \quad \frac{w_q P(q, r)}{w_r} \in \mathbb{D}_{\geq 0}. \quad (50)$$

Indeed, a positive edge satisfies $P(q, r) = p_{\mathcal{M}}(\gamma r) / p_{\mathcal{M}}(\gamma)$ for any positive trajectory γ ending at q , so every positive transition is rational.

Fix an odd prime p . For a positive rational a/b in lowest terms, define $v_p(a/b)$ as the exponent of p in a minus its exponent in b . Let

$$\Gamma(r) = \{\gamma = (x_0, \dots, x_n) : n \geq 0, x_n = r, p_{\mathcal{M}}(\gamma) > 0\} \quad (51)$$

be the set of positive-probability finite trajectories ending at r , and define

$$0 \leq \alpha_r^p = \min_{\gamma \in \Gamma(r)} v_p(p_{\mathcal{M}}(\gamma)). \quad (52)$$

Equivalently, add a source $*$ with an edge $* \rightarrow q$ of weight $v_p(\mu(q))$ whenever $\mu(q) > 0$, and give each positive edge $q \rightarrow r$ weight $v_p(P(q, r))$. Then α_r^p is the shortest-path distance from $*$ to r . Path-dyadicity rules out negative cycles and makes this distance nonnegative, so the minimum is attained.

If $P(q, r) > 0$, append the edge $q \rightarrow r$ to a trajectory attaining α_q^p . This gives

$$\alpha_r^p \leq \alpha_q^p + v_p(P(q, r)). \quad (53)$$

If $\mu(r) > 0$, the length-zero trajectory (r) similarly gives

$$\alpha_r^p \leq v_p(\mu(r)). \quad (54)$$

Now set

$$w_r = \prod_{p \text{ odd prime}} p^{\alpha_r^p}. \quad (55)$$

This product is finite because the finitely many nonzero entries of μ and P involve only finitely many odd primes.

For every odd prime p , the two preceding inequalities give

$$v_p\left(\frac{\mu(r)}{w_r}\right) = v_p(\mu(r)) - \alpha_r^p \geq 0 \quad (56)$$

when $\mu(r) > 0$, and

$$v_p\left(\frac{w_q P(q, r)}{w_r}\right) = \alpha_q^p + v_p(P(q, r)) - \alpha_r^p \geq 0 \quad (57)$$

when $P(q, r) > 0$. A positive rational is dyadic exactly when its valuation at every odd prime is nonnegative; zero entries are dyadic as well. Thus the weights satisfy equation (50).

Split state q into the fiber $F_q = \{(q, 1), \dots, (q, w_q)\}$ and let $\tilde{Q} = \bigsqcup_q F_q$. Choose s so that

$$m_{q,r} = 2^s \frac{w_q P(q, r)}{w_r} \quad (58)$$

is an integer for every q, r . For each q , choose an integer matrix $C^{(q)}$ with rows indexed by F_q and columns by $\tilde{Q}$, such that every row sums to 2^s and every column in F_r sums to $m_{q,r}$. Such a matrix exists because both total sums equal $2^s w_q$.

For $h \in F_q$, define

$$\tilde{\mu}(h) = \frac{\mu(q)}{w_q} \quad (h \in F_q), \quad \tilde{P}(h, h') = \frac{C^{(q)}(h, h')}{2^s}. \quad (59)$$

Both are dyadic. We prove by induction on n that, for every positive trajectory $\gamma = (x_0, \dots, x_n)$ and every $h \in F_{x_n}$,

$$\Pr[\phi(\tilde{X}_0) = x_0, \dots, \phi(\tilde{X}_{n-1}) = x_{n-1}, \tilde{X}_n = h] = \frac{p_{\mathcal{M}}(\gamma)}{w_{x_n}}. \quad (60)$$

For $n = 0$, this is the definition of $\tilde{\mu}$. Suppose it holds for a trajectory γ ending at q . For every positive extension γr and every $h' \in F_r$, the column-sum condition on $C^{(q)}$ gives

$$\sum_{h \in F_q} \frac{p_{\mathcal{M}}(\gamma)}{w_q} \tilde{P}(h, h') = \frac{p_{\mathcal{M}}(\gamma)}{w_q} \sum_{h \in F_q} \frac{C^{(q)}(h, h')}{2^s} \quad (61)$$

$$= \frac{p_{\mathcal{M}}(\gamma)}{w_q} \frac{m_{q,r}}{2^s} \quad (62)$$

$$= \frac{p_{\mathcal{M}}(\gamma) P(q, r)}{w_r} = \frac{p_{\mathcal{M}}(\gamma r)}{w_r}. \quad (63)$$

This proves the induction step. Summing over $h \in F_{x_n}$ gives $p_{\mathcal{M}}(\gamma)$, proving the projection claim. $\square$

Corollary 4.5 (Exact sampling characterization). *A fixed finite Markov chain has a randomized-AC⁰ trajectory sampler for all horizons if and only if it is path-dyadic.*

Proof. A sampler using r_n fair bits assigns every atom an integer multiple of 2^{-r_n} , proving necessity. For sufficiency, apply theorem 4.4 followed by theorem 4.3 and project each hidden output coordinate through the fixed map ϕ . $\square$

4.3 From Markov Trajectories to Regular-Language Pairs

The preceding characterization applies directly to a DFA driven by a uniform random input.

Corollary 4.6 (Binary regular-language sampling). *Every fixed binary regular language $L \subseteq \{0, 1\}^*$ has a randomized-AC⁰ sampler for $\text{Pair}_{L,n}$.*

Proof. Fix a DFA $\mathcal{A} = (Q, \{0, 1\}, \delta, q_0, F)$ for L and define

$$A(q, r) = \{a \in \{0, 1\} : \delta(q, a) = r\}, \quad P(q, r) = \frac{|A(q, r)|}{2}. \quad (64)$$

Thus

$$P(q, r) \in \left\{0, \frac{1}{2}, 1\right\} \subseteq \mathbb{D}_{\geq 0}. \quad (65)$$

Hence the chain is transition-dyadic and therefore path-dyadic.

With $\mu = e_{q_0}$, corollary 4.5 samples its state trajectory

$$(Y_0, Y_1, \dots, Y_n), \quad Y_0 = q_0. \quad (66)$$

Conditioned on this path, independently sample X_{i+1} by

$$\Pr[X_{i+1} = a \mid Y_i, Y_{i+1}] = \frac{\mathbf{1}[\delta(Y_i, a) = Y_{i+1}]}{\sum_{b \in \{0, 1\}} \mathbf{1}[\delta(Y_i, b) = Y_{i+1}]} \in \{0, 1/2, 1\}. \quad (67)$$

for $0 \leq i < n$ and $a \in \{0, 1\}$. Each coordinate uses at most one fresh fair bit. Then $X = (X_1, \dots, X_n) \sim U_n$ and

$$C = \mathbf{1}[Y_n \in F] = f_L(X). \quad (68)$$

Thus (X, C) has distribution $\text{Pair}_{L,n}$, and the postprocessing is in randomized- AC^0 . $\square$

Remark 4.7 (Why a sequential DFA becomes shallow). The direct recurrence $Y_{i+1} = \delta(Y_i, X_{i+1})$ is sequential. The sampler does not evaluate this recurrence on the original fair bits. It first applies a measure-preserving cube reparameterization whose induced block transformations generate an aperiodic monoid. Aperiodicity is the bridge that makes every prefix state simultaneously computable in AC^0 .

4.4 Two Rounds with Revision

The randomized circuit from corollary 4.6 can be embedded into two DLM rounds without adding output positions.

Theorem 4.8 (Two-round revision sampling). *For every fixed binary regular language L and every $n \geq 1$, a randomized- AC^0 DLM with revision samples $\text{Pair}_{L,n}$ in at most two rounds.*

Proof. Fix a DFA $\mathcal{A} = (Q, \{0, 1\}, \delta, q_0, F_{\mathcal{A}})$ for L . The construction in corollary 4.6 uses at most n fair bits for the DFA trajectory. In round one, sample the first n coordinates independently and uniformly, and set the label coordinate to a fixed placeholder. In round two, map the first-round bits to the state trajectory $(Y_0, \dots, Y_n)$ and update each input coordinate according to equation (67). These updates are coordinatewise once the trajectory is fixed. Set the label to $\mathbf{1}[Y_n \in F_{\mathcal{A}}]$. The output law is $\text{Pair}_{L,n}$. $\square$

For parity, one round is impossible: a one-round update from a fully masked state is a product distribution, whereas uniform odd parity on $n + 1$ bits is not a product distribution. Hence the revision complexity of parity is exactly two.

5 No-Revision Round Complexity

Revision destroys the irreversible trace structure used in lower bounds. In the no-revision model, however, every completed output fixes the symbols that must have appeared whenever a position was first exposed.

The mask profile itself can carry a constant-size monoid summary while the ordinary output coordinates remain the only available workspace:

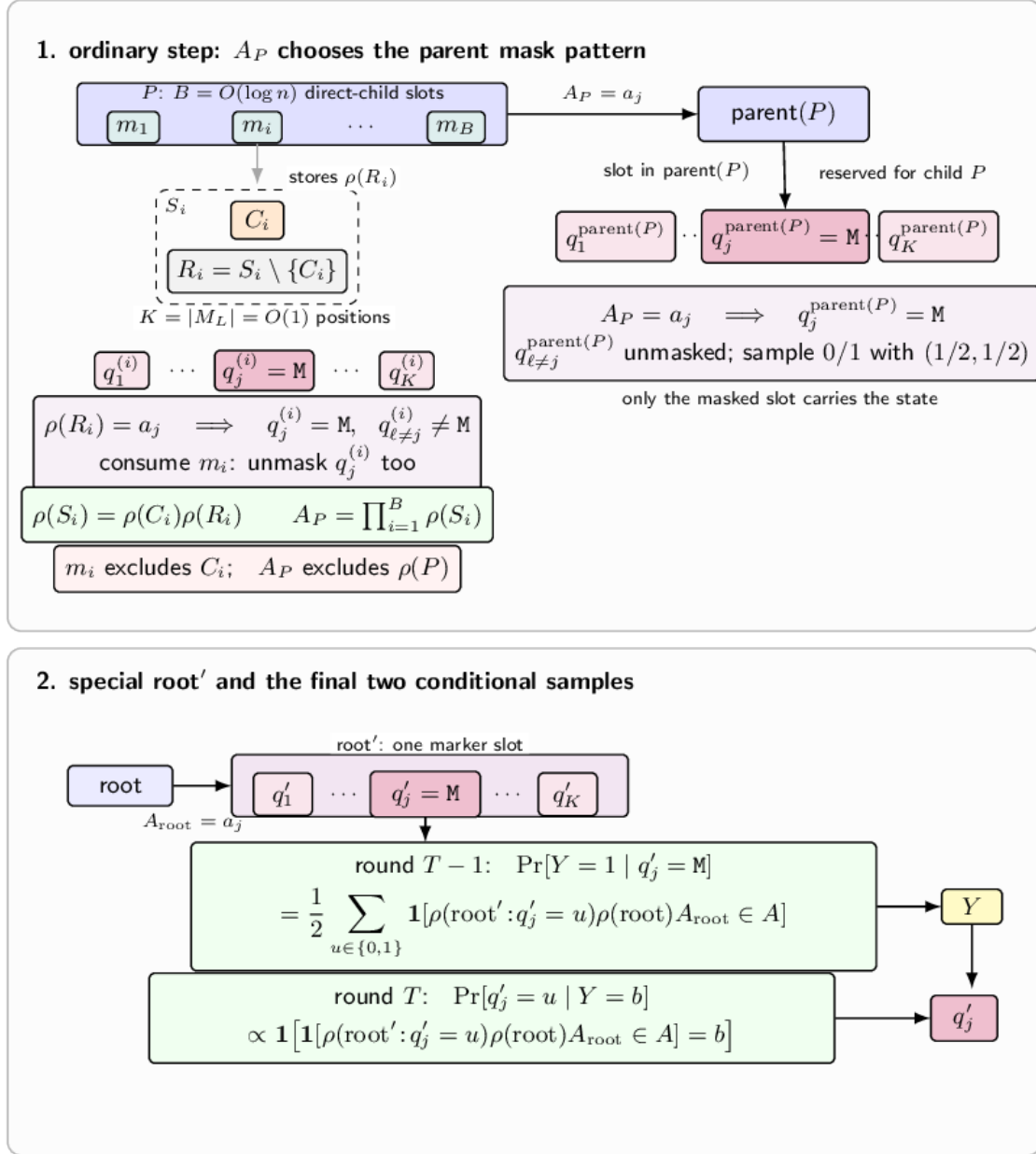


Figure 1: No-revision product-tree summaries stored in the mask profile of ordinary output coordinates.

Theorem 5.1 (No-revision bounds). *In the randomized- AC^0 no-revision model, every fixed binary regular language has an $O(\log n / \log \log n)$ -round sampler for $\text{Pair}_{L,n}$. Under the exact finite-fair-bit local-kernel model above, if $L \notin \text{AC}^0$, then every such sampler requires $\Omega(\log n / \log \log n)$ rounds.*

5.1 Upper bound: a high-arity monoid tree

Use the standard finite-monoid product tree for regular languages. Lay the tree out in depth-first index order, so that each subtree occupies a contiguous coordinate interval, including the internal positions assigned to its root. Fix a finite monoid M_L , a morphism $\rho : \{0, 1\}^* \rightarrow M_L$, and an accepting set $F_{\text{acc}} \subseteq M_L$ such that

$$f_L(x) = \mathbf{1}[\rho(x) \in F_{\text{acc}}]. \quad (69)$$

Equivalently, $\rho(w)$ records the DFA transition induced by a substring w ; applied to the start state, it gives the DFA state after reading w .

Enumerate

$$M_L = \{a_1, \dots, a_K\}, \quad K = |M_L| = O(1), \quad (70)$$

and take a branching factor $B = \Theta(\log n)$. The sampler uses a B -ary product tree over the first n output coordinates. A marker slot is not an extra alphabet symbol: it is a constant-size block of ordinary output coordinates whose mask pattern carries a monoid value. In the convention used below, one coordinate in the slot is masked, and the index of that masked coordinate encodes the value.

At an ordinary node P , write its direct-child subtrees in string order as $S_1, \dots, S_B$. For each child subtree S_i , let C_i be the root coordinate of that subtree and let

$$R_i = S_i \setminus \{C_i\}. \quad (71)$$

The slot m_i reserved for child S_i consists of K positions $q_1^{(i)}, \dots, q_K^{(i)}$. It stores only the remainder value $\rho(R_i)$, not the root coordinate C_i , by the one-mask convention

$$\rho(R_i) = a_j \iff q_j^{(i)} = \mathbf{M}, \quad q_{\ell \neq j}^{(i)} \neq \mathbf{M}. \quad (72)$$

When this child slot is consumed, the slot's still-masked coordinate is also unmasked and sampled as a fair bit. After the coordinate C_i is sampled, the predictor can compute

$$\rho(S_i) = \rho(C_i)\rho(R_i). \quad (73)$$

The value passed from P to $\text{parent}(P)$ is then

$$A_P = \prod_{i=1}^B \rho(S_i). \quad (74)$$

As in figure 1, A_P still excludes $\rho(P)$ itself. If $A_P = a_j$, then in the slot of $\text{parent}(P)$ reserved for child P , the next round creates the mask pattern

$$q_j^{\text{parent}(P)} = \mathbf{M}, \quad q_{\ell \neq j}^{\text{parent}(P)} \neq \mathbf{M}, \quad (75)$$

sampling every newly unmasked $q_{\ell \neq j}^{\text{parent}(P)}$ as a fair bit. At the same time, the consumed marker positions inside P are also unmasked, including the unique previously masked coordinate in each consumed child slot. Since K is constant and $B = \Theta(\log n)$, the map

$$(\rho(S_1), \dots, \rho(S_B)) \mapsto A_P \quad (76)$$

is an AC^0 table lookup. One such upward step is performed in parallel at every node on the current tree level. The height is therefore

$$O(\log_B n) = O\left(\frac{\log n}{\log \log n}\right). \quad (77)$$

At the top, add the special node root' . Unlike an ordinary node, root' has only one marker slot $q'_1, \dots, q'_K$. After the root coordinate has been sampled, the upward pass stores the root aggregate A_{root} in this slot:

$$A_{\text{root}} = a_j \iff q'_j = \mathbf{M}, \quad q'_{\ell \neq j} \neq \mathbf{M}. \quad (78)$$

At this point the only masks are q'_j and the final output coordinate Y . The hidden bit q'_j is still an ordinary prefix coordinate and should be fair in the final output. The last two rounds sample Y first and then sample this hidden bit from the correct posterior:

$$\Pr[Y = 1 \mid q'_j = \mathbf{M}] = \frac{1}{2} \sum_{u \in \{0,1\}} \mathbf{1}[\rho(\text{root}': q'_j = u) \rho(\text{root}) A_{\text{root}} \in F_{\text{acc}}], \quad (79)$$

then, after $Y = b$ is visible,

$$\Pr[q'_j = u \mid Y = b] \propto \mathbf{1}[\mathbf{1}[\rho(\text{root}': q'_j = u) \rho(\text{root}) A_{\text{root}} \in F_{\text{acc}}] = b]. \quad (80)$$

The factor $1/2$ in the first formula is the prior probability of the hidden fair bit $q'_j = u$; the second formula is the corresponding posterior after observing $Y = b$. Thus every prefix coordinate, including the marker coordinate resolved at the end, is uniform, and the final coordinate equals $Y = f_L(X)$.

5.2 Lower bound: support tracing

Suppose there is a D -round no-revision sampler for $\text{Pair}_{L,n}$. Its support contains one output pair for each input:

$$\Pr[Z = (x, b)] = \begin{cases} 2^{-n}, & b = f_L(x), \\ 0, & b \neq f_L(x), \end{cases} \quad (81)$$

where Z denotes the sampler output. Because F is deterministic and decoded bits never change, each output determines a unique complete trace. Every positive trace therefore has probability 2^{-n} . Write each positive dyadic local factor as $m_j/2^{r_j}$. Since their product is 2^{-n} , every m_j is a power of two, so every factor has the form 2^{-a_j} .

Now consider a reachable binary local choice. If both outcomes have positive probability, completing each branch to a positive output shows that $p = 2^{-a}$ and $1 - p = 2^{-b}$ for integers $a, b \geq 1$. The identity $2^{-a} + 2^{-b} = 1$ forces $a = b = 1$. Thus every reachable local binary distribution is $(1, 0)$, $(0, 1)$, or $(1/2, 1/2)$. In particular, every positive local branch has probability at least $1/2$.

Therefore the candidate output $(x, 1)$ has positive probability if and only if $x \in L$. We can test this support event by tracing the sampler along the candidate output. At each no-revision step, use the sampler's unmasking policy F and predictor p ; if a required local value has probability 0, reject, and otherwise estimate positivity by repeated independent draws from the same local predictor.

The rigidity argument above shows that every positive local branch on such a trace has probability at least $1/2$. Enumerate all reachable positive local judgments (s, i, b) : there are at most $2(n+1)3^{n+1} = O(n3^n)$ of them. Using the same k fresh random blocks for every such judgment, k independent samples miss a fixed positive judgment with probability at most 2^{-k} . Taking

$$k = \lceil \log_2(2(n+1)3^{n+1}) \rceil + 1 = O(n) \quad (82)$$

gives

$$\Pr[\text{some reachable positive judgment is missed}] \leq 2(n+1)3^{n+1} 2^{-k} < 1. \quad (83)$$

Algorithm 5 SupportTest: deciding whether the candidate pair $(x, 1)$ lies in the sampler’s support

```

1: Input:  $x \in \{0, 1\}^n$ 
2: set  $x' \leftarrow (x, 1)$ 
3: set  $s \leftarrow \mathbb{M}^{n+1}$ 
4: for  $t = 1, \dots, D$  do
5:    $S \leftarrow F(s)$ 
6:   if  $S \not\subseteq \{i : s_i = \mathbb{M}\}$  then
7:     return reject
8:   for all  $i \in S$  do
9:     draw  $k$  independent samples from  $p_i(\cdot \mid s)$ 
10:    if none of the  $k$  samples equals  $x'_i$  then
11:      return reject
12:    set  $s_i \leftarrow x'_i$ 
13: return accept iff  $s = x'$ 

```

Hence one fixed seed, of length $k \cdot \text{poly}(n) = \text{poly}(n)$, makes every reachable positive decision correct, while zero-probability decisions are never accepted. Hardwiring this seed turns the unrolled D -round procedure into a deterministic AND/OR/NOT recognizer for L of depth $O(D)$. The regular-language lower bound therefore gives

$$D = \Omega\left(\frac{\log n}{\log \log n}\right) \quad (84)$$

whenever $L \notin \text{AC}^0$.

It remains to connect recognition depth to regular-language structure. The stable-monoid characterization and standard depth lower bounds for regular languages outside AC^0 imply that such a language cannot have a polynomial-size randomized recognizer of depth $o(\log n / \log \log n)$. Consequently the hardwired recognizer above yields the lower bound in theorem 5.1. Combining both directions gives the following separation.

Corollary 5.2 (Parity separation). *For $L = \{x : \sum_i x_i \equiv 0 \pmod{2}\}$, revision complexity is exactly two, whereas no-revision complexity is $\Theta(\log n / \log \log n)$.*

Proof. Since $L \notin \text{AC}^0$, theorem 5.1 gives the no-revision claim, and theorem 4.8 gives a two-round revision sampler. A one-round update from $\mathbb{M}^{n+1}$ is a product distribution, whereas the uniform odd-parity distribution on $n + 1$ bits is not a product distribution. Thus one round is impossible, and the revision complexity is exactly two. $\square$

6 Transformer Realization

We adapt the method of Li et al. [5]. Fix a constant-depth, unmasked bidirectional Transformer with one-hot positional encodings, polynomial embedding width, no padding, and constant-bit fixed-point arithmetic with rounding after every operation. To start, write

$$\mathbb{F}_s = \mathbb{F}_{0,s}, \quad B_s = \max \mathbb{F}_s = 2^s - 2^{-s}, \quad (85)$$

as our finite-precision model for a constant $s \geq 2$. Thus B_s is the largest representable number, and $[\cdot]_s$ denotes rounding into $\mathbb{F}_s$. The value B_s enters through two opposite overflow regimes: saturation $[\exp(B_s)]_s = B_s$ makes attention copy exactly, while underflow $[\exp(-B_s)]_s = 0$ produces exact zero probabilities.

At each round, the Transformer backbone first computes a positionwise logit vector. For the predictor, apply rounded softmax to obtain probabilities $p_1, \dots, p_{|V|}$ in a fixed token order. Draw z uniformly from the finite-precision grid in $[0, 1)$ and output the first token j such that

$$z < \sum_{i=1}^j p_i. \quad (86)$$

The draws are independent across updated positions. Once the logits are given, this inverse-CDF decoding is a randomized- AC^0 computation because the vocabulary and precision are constant. The no-revision policy instead computes logits over the two actions and takes their argmax, which is an AC^0 computation once the logits are given.

Li et al. prove the equivalence between AC^0 and next-token prediction by constant-depth, polynomial-width, constant-precision Transformers, with the predicted token selected by argmax [5]. Their statement uses a decoder-only Transformer, but both directions extend directly to our bidirectional model: the original construction gathers the input at the last position, whereas bidirectional attention allows every position to gather it.

Equivalently, a positionwise finite-precision logit vector is computable by this Transformer model if and only if it is AC^0 -computable. The forward direction follows directly from Li et al. Conversely, write a desired logit as

$$z = \sum_i z_i 2^i, \quad z_i \in \{-1, 0, 1\}. \quad (87)$$

If the logit is AC^0 -computable, each digit z_i is AC^0 -computable. The Transformer construction stores these digits in the hidden dimension, and the final output projection matrix assigns weight 2^i to digit z_i , thereby reassembling z exactly. This proves the logit-level equivalence needed for both transfers below.

6.1 Lower-bound transfer

The Transformer computes AC^0 logits, after which the predictor applies randomized- AC^0 decoding and the policy applies AC^0 argmax. Thus the resulting DLM satisfies the randomized- AC^0 regime of section 3, so the no-revision lower bound in theorem 5.1 transfers unchanged.

6.2 Upper-bound realization

For the upper bounds, every choice among the required policy and predictor logits is an AC^0 function of the current state, so the logit-level equivalence gives a Transformer realization. The policy uses $(0, -B_s)$ for hold and $(-B_s, 0)$ for unmask. The predictor constructions use the following positionwise logit vectors over $\{0, 1\} \cup \{\mathbf{M}\}$:

$$(0, -B_s, -B_s), \quad (-B_s, 0, -B_s), \quad (0, 0, -B_s) \quad (88)$$

whose rounded softmax is exactly

$$(1, 0, 0), \quad (0, 1, 0), \quad (1/2, 1/2, 0) \quad (89)$$

Indeed, $[\exp(-B_s)]_s = 0$ and $[\exp(0)]_s = 1$, and the denominator $1 + 1 = 2$ is exactly representable when $s \geq 2$, so the fair case is exactly $1/2$. Independent decoding therefore realizes every predictor used in the two-round revision construction and the no-revision product-tree construction.

The two transfers give the following formal statement.

Theorem 6.1 (Transformer round bounds). *Under the Transformer model specified in this section, every fixed binary regular language has a revision sampler using at most two rounds and a no-revision sampler using $O(\log n / \log \log n)$ rounds. If $L \notin \text{AC}^0$, every no-revision sampler requires $\Omega(\log n / \log \log n)$ rounds. In particular, parity has revision complexity exactly two and no-revision complexity $\Theta(\log n / \log \log n)$.*

Proof. The upper bounds follow from the logit realization above and theorems 4.8 and 5.1. The no-revision lower bound follows from the lower-bound transfer. For parity, one revision round still produces a product distribution from the fully masked initial state, so two rounds are necessary. $\square$

7 Discussion, Limitations, and Future Work

Exact versus approximate sampling. Exactness is central to both the dyadic characterization and the support reduction. Approximate sampling may bypass the arithmetic obstruction and can interact differently with circuit lower bounds. Extending the round separation to small total-variation error is open.

Fixed finite objects. The DFA and Markov chain are fixed while n grows. State count, transition denominators, block length, hidden-fiber size, and monoid tables are constants hidden in the circuit family. Growing automata, growing moduli, or a Markov chain supplied as part of the input require new uniformity and size analyses.

Workspace. The DLM output has no extra chain-of-thought or padding positions. Revision reuses the output coordinates, while the no-revision construction stores constant-size summaries in mask profiles. Allowing polynomially many scratch positions changes the resource tradeoff and connects more directly to general parallel-circuit simulations.

Research directions. Natural next steps include an approximate analogue of path-dyadicity, uniform reconstruction algorithms for the aperiodic lift, quantitative bounds on the hidden-state blowup, growing-state Markov chains, and lower bounds for revision when the target family is not fixed regular. On the modeling side, it remains important to compare the idealized conditional-independence update rule with the numerical behavior of trained diffusion language models.

8 Conclusion

The project identifies a common mechanism behind exact finite-state trajectory sampling and parallel generation by diffusion language models. Path-dyadicity is exactly the arithmetic condition for a fixed finite Markov trajectory to be sampled from fair bits; a finite dyadic lift and aperiodic cube reparameterization turn that arithmetic condition into a constant-depth circuit. A uniformly random word driving a DFA is a particularly clean transition-dyadic chain, so every fixed binary regular-language input–output pair has an exact randomized- AC^0 sampler. Revision lets a DLM realize this sampler in two rounds. Without revision, information must instead propagate through immutable mask profiles, producing the $\Theta(\log n / \log \log n)$ behavior for languages outside AC^0 . The resulting separation makes precise, in a finite-state setting, how writable intermediate state can reduce the sequential depth of exact parallel generation.

AI disclosure. I used generative AI tools to assist with drafting and revising prose, organizing the presentation, and checking LaTeX and terminology consistency.

References

- [1] David A. Mix Barrington, Kevin Compton, Howard Straubing, and Denis Thérien. Regular languages in NC^1 . *Journal of Computer and System Sciences*, 44(3):478–499, 1992. doi: 10.1016/0022-0000(92)90014-A.
- [2] Massimiliano Goldwurm, Beatrice Palano, and Massimo Santini. On the circuit complexity of random generation problems for regular and context-free languages. In *STACS 2001*, volume 2010 of *Lecture Notes in Computer Science*, pages 181–192, 2001. doi: 10.1007/3-540-44693-1_27.
- [3] Haozhe Jiang, Nika Haghtalab, and Lijie Chen. Diffusion language models are provably optimal parallel samplers. In *International Conference on Learning Representations*, 2026.
- [4] Jürgen Jost, Martin Kell, and Christian S. Rodrigues. Representation of markov chains by random maps: Existence and regularity conditions. *Electronic Communications in Probability*, 20:1–10, 2015. doi: 10.1214/ECP.v20-3771.
- [5] Zhiyuan Li, Hong Liu, Denny Zhou, and Tengyu Ma. Chain of thought empowers transformers to solve inherently serial problems. In *International Conference on Learning Representations*, 2024.

- [6] William Merrill, Ashish Sabharwal, and Noah A. Smith. Saturated transformers are constant-depth threshold circuits. *Transactions of the Association for Computational Linguistics*, 10:843–856, 2022. doi: 10.1162/tacl_a_00493.
- [7] Anej Svete and Ashish Sabharwal. On the reasoning abilities of masked diffusion language models. *International Conference on Learning Representations*, 2026.
- [8] Hermann Thorisson. *Coupling, Stationarity, and Regeneration*. Probability and Its Applications. Springer, New York, 2000. doi: 10.1007/978-1-4612-1236-2.
- [9] Emanuele Viola. The complexity of distributions. *SIAM Journal on Computing*, 41(1):191–218, 2012. doi: 10.1137/100814998.
- [10] Emanuele Viola. Sampling lower bounds: Boolean average-case and permutations. *SIAM Journal on Computing*, 49(1):119–137, 2020. doi: 10.1137/18M1198405.
- [11] Kouji Yano and Kenji Yasutomi. Random walks driven by markov chains and road coloring. *Journal of Theoretical Probability*, 24:720–733, 2011. doi: 10.1007/s10959-010-0306-5.